MITRC

MODERN INSTITUTE OF TECHNOLOGY AND RESEARCH CENTRE

Name of Faculty: Dr. J.R.Arun kumar

Subject & Subject Code: Natural Language Processing (**6CS5-13**)

Semester: VI **Session:** 2023-24(Even Sem.)

Unit No.: I **Date of Submission:** 13/02/2024

Sl.No.	Topics Covered	Page No.	Total pages
1	Introduction: A computational framework for natural language,	1-7	07
2	description of English or an Indian language in the framework,	8-12	05
3	lexicon, algorithms and data structures for implementation of the framework	13-14	02
4	Finite state automata	15-20	06
5	Applications like machine translations	21-23	03

References Book:

Natural Language understanding by James Allen, Pearson Education 2008
NLP: A Paninian Perspective by Akshar Bharati, Vineet Chaitanya, and Rajeev Sangal, Prentice Hal

FACULTY HOD 13/2/24 DEAN ACADEMICS

UNIT-1 (INTRODUCTION TO NLP)

Natural Language Processing

LECTURE NO:-1

INTRODUCTION NLP:

Natural language processing (NLP) is a machine learning technology that gives computers the ability to interpret, manipulate, and comprehend human language. Organizations today have large volumes of voice and text data from various communication channels like emails, text messages, social media newsfeeds, video, audio, and more. They use NLP software to automatically process this data, analyze the intent or sentiment in the message, and respond in real time to human communication.

WHY IS NLP IMPORTANT?

Natural language processing (NLP) is critical to fully and efficiently analyze text and speech data. It can work through the differences in dialects, slang, and grammatical irregularities typical in day-to-day conversations.

Companies use it for several automated tasks, such as to:

- Process, analyze, and archive large documents
- Analyze customer feedback or call center recordings
- Run chatbots for automated customer service
- Answer who-what-when-where questions
- Classify and extract text

You can also integrate NLP in customer-facing applications to communicate more effectively with customers. For example, a chatbot analyzes and sorts customer queries, responding automatically to common questions and redirecting complex queries to customer support. This automation helps reduce costs, saves agents from spending time on redundant queries, and improves customer satisfaction.

HOW DOES NLP WORK?

Natural language processing (NLP) combines computational linguistics, machine learning, and deep learning models to process human language.

COMPUTATIONAL LINGUISTICS

Computational linguistics is the science of understanding and constructing human language models with computers and software tools. Researchers use computational linguistics methods, such as

syntactic and semantic analysis, to create frameworks that help machines understand conversational human language. Tools like language translators, text-to-speech synthesizers, and speech recognition software are based on computational linguistics.

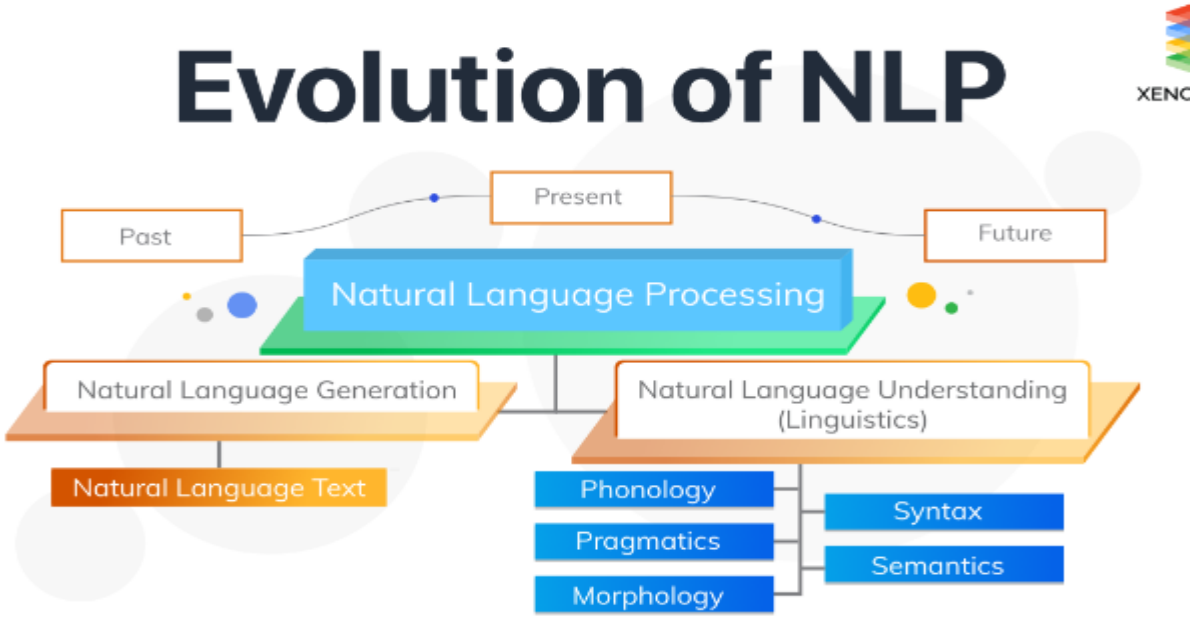
MACHINE LEARNING

Machine learning is a technology that trains a computer with sample data to improve its efficiency. Human language has several features like sarcasm, metaphors, variations in sentence structure, plus grammar and usage exceptions that take humans years to learn. Programmers use machine learning methods to teach NLP applications to recognize and accurately understand these features from the start.

DEEP LEARNING

Deep learning is a specific field of machine learning which teaches computers to learn and think like humans. It involves a neural network that consists of data processing nodes structured to resemble the human brain. With deep learning, computers recognize, classify, and co-relate complex patterns in the input data.

A COMPUTATIONAL FRAMEWORK FOR NATURAL LANGUAGE



Simply, a language can be understood as a group of rules or symbols. These symbols are integrated and then used for transmitting and broadcasting the information. Here, rules are applied to suppress

the signs. Natural Language Processing is divided into sub-areas, i.e., Natural Language Generation and Natural Language Understanding, which are, as the name suggests, associated with the generation and understanding of the text. The following chart broadly shows these points. Don't get confused by these new terms, such as Phonology, Pragmatics, Morphology, Syntax, and Semantics. Let's explore these in a very brief manner-

- **Phonology -** This science helps to deal with patterns present in the sound and speeches related to the sound as a physical entity.
- **Pragmatics -** This science studies the different uses of language.
- **Morphology -** This science deals with the structure of words and the systematic relations between them.
- **Syntax -** This science deals with the structure of the sentences.
- **Semantics -** This science deals with the literal meaning of words, phrases, and sentences.



2.1 Introduction to Language Structure

In this section, we introduce the reader to language structure. As discussed in Sec. 1.2, we are primarily concerned with the communication function of natural language; in particular, how information is coded in a natural language string[1] and how it is retrieved from the string by a reader. The purpose of identifying structure is that it will establish a relation between information[2] and the string that codes it. Before we can write grammars and design NLP systems that use the grammars, we need to sharpen our intuitions about language. This, then, is the purpose of this section.

The basic units in a written string are words. A word is a continuous sequence of alphabetic characters (possibly including the hyphen).[3] It stands for some information. Further, appropriate sequence of words in a string can be grouped together in a word group. What this means is that for such a group of words, there is a pattern or a rule which helps define the group. Information contained in a word group usually corresponds in some systematic way to the information contained in the constituent words. In other words, information is compositional in a group. Similarly, a word group can be further combined together with other word groups or words to yield larger groups. Here again, the information contained in the larger group systematically corresponds to the information contained in the constituents.

When two or more items get grouped into a larger unit, a structure is formed, which contains the items and the relationship among them. The information of the larger unit, usually depends on the structure, that is, the constituent items and their inter-relations.

To keep the structure minimal, we introduce only one kind of structure, call it *modifier-modified structure*. There will usually be two items in a structure called the *head* and the *modifier*. [4] For example, in the following Hindi string:

```
sheitaana  laDake
naughty    boys
```

'naughty' is the modifier and 'boys' is the head. Similarly, in the following:

```
lohe  kaa   pampa
iron  reln. pump
(iron pump.)
```

pump is the head, and iron is the modifier.[5] Again, in the following:

```
paanii  kaa    pampa
water   reln.  pump
(water pump)
```

pump is the head, and water is a modifier. Note that properties of the head are inherited by the group. Thus, 'water pump' is a pump, it is not water.[6] Linguistically, we see that the grammatical properties of the head are inherited by the structure. Thus, in the sentence:

```
Iron pumps are heavy.
```

Arguments of certain verbs are verbs. Consider the sentence:

```
laDake ne kahaa ki usane patanga uDaayii.
boy  nom. said that he   kite    flew.
(The boy said that he flew a kite.)
```

Here the argument of 'say' is an entire sentence 'he flew a kite'.

Diagramatically it is shown in Figure 2.2. As before, a child node is

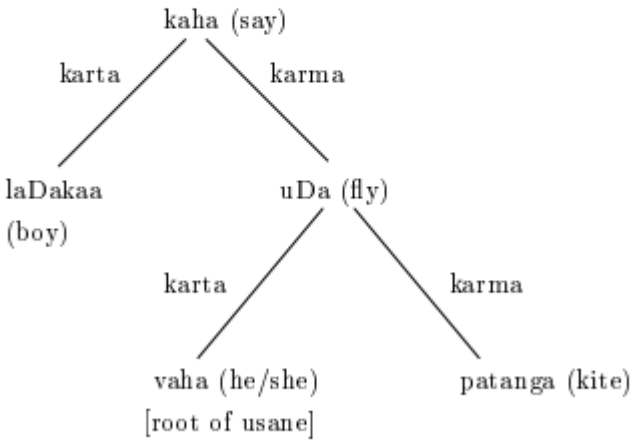


Figure 2.2: A verb (uDa) as argument of another verb (kaha)

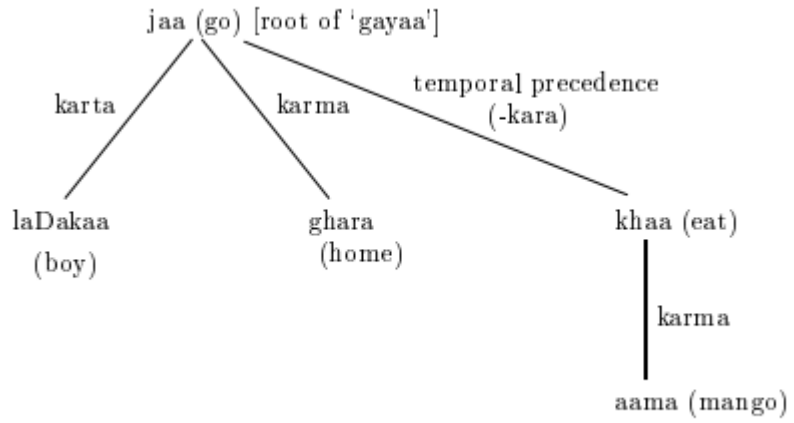


Figure 2.4: A verb-verb modification

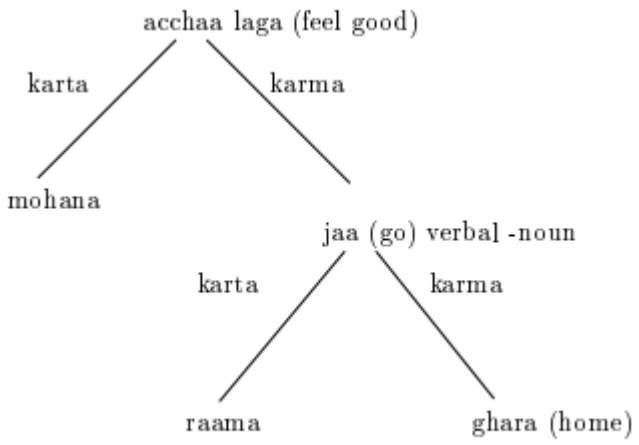


Figure 2.5: Verbal noun

made is based on the karaka relations. For us the parse structure for a sentence consists primarily of the verbal groups and the nominals in the sentence, and the karaka relations among them. Non-karaka relations may also be present due to adjectives, hetu (purpose) relation, relational words like 'door' (far) and 'paasa' (near), etc. Justification for the choice of parse structure is given in the later chapters (for example, Chap. 5.). Here, we present the parse structure without discussing why it has been chosen.

Take the following sentence as an example:

```
kisaana kheta jotataa hei
farmer farm  ploughs
(Farmer ploughs the farm.)
```

It will be assigned the following parse structure:

```
jota (plough)
  karta: kisaana (farmer)
  karma:  kheta (farm)
```

where 'kisaana' is the karta and 'kheta' is the karma of 'jota'. This is shown diagrammatically in Figure 2.6.

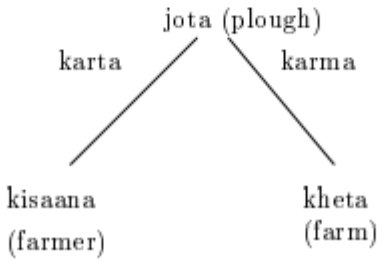


Figure 2.6: A parse structure

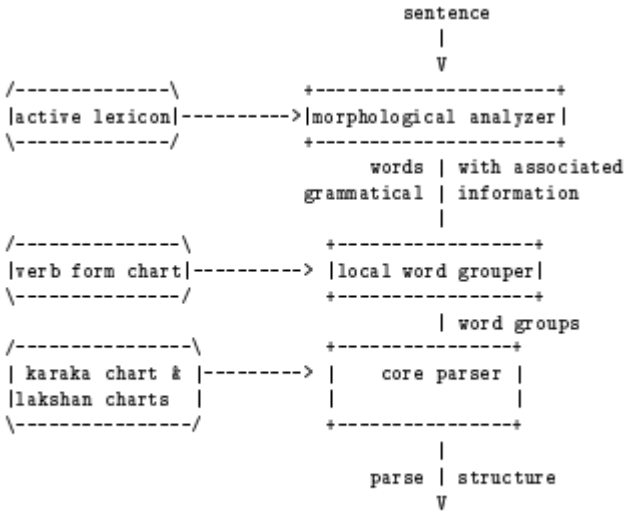


Figure 2.7: Structure of the parser

UNIT-1 (INTRODUCTION TO NLP)

LECTURE NO:-2

Natural Language Processing

Description of English or an Indian language in the framework,

ENGLISH SYNTAX

The order in which words and phrases occur matters. Syntax is the set of conventions of language that specify whether a given sequence of words is well-formed and what functional relations, if any, pertain to them. For example, in English, the sequence "cat the mat on" is not well-formed. To keep the set conventions manageable, and to reflect how native speakers use their language, syntax is defined hierarchically and recursively. The structures include words (which are the smallest well-formed units), phrases (which are legal sequences of words), clauses and sentences (which are both legal sequences of phrases). Sentences can be combined into compound sentences using conjunctions, such as "and".

The categories of words used for NLP are mostly similar to those used in other contexts, but in some cases they may be different from how you were taught when you were learning the grammar of English. One of the challenges faced in NLP work is that terminology for describing syntax has evolved over time and differs somewhat across disciplines. If one were to ask how many categories of words are there, the writing center at a university might say there are eight different types of parts of speech in the English language: noun, pronoun, verb, adjective, adverb, preposition, conjunction, and interjection[1]. The folks who watched Schoolhouse Rock also were taught there were eight, but a slightly different set[2]. By contrast, the first published guideline for annotating part of speech created by linguists used eighty different categories[3]. Most current NLP work uses around 35 labels for different parts of speech, with additional labels for punctuation.

The conventions for describing syntax arise from two disciplines: studies by linguists, going back as far as 8th century BCE by the first Sanskrit grammarians, and work by computational scientists, who have standardized and revised the labeling of syntactic units to better meet the needs of automated processing. What forms a legal constituent in a given language was once determined qualitatively and empirically: early linguists would review written documents, or interview native speakers of language, to find out what phrases native speakers find acceptable and what phrases or words can be substituted for one another and still be considered grammatical. This technique is still useful today as a means of verifying the syntactic labels of rarely seen expressions. For example, in the sentence "I would rather deal with the side effects of my medication", one might wonder if "with" is part of a complex verb "deal with" or

it acts as a preposition, which is a function word more associated with the noun phrase "the side effects". The fact that we can substitute the verb "tolerate" for "deal with" is evidence that "deal with" is a single entity.

WORD TYPES AND FEATURES

In English, we typically think of the word as the smallest unit. However, trained linguists make some finer grained distinctions. For example, linguists use the term lemma, or root, or base form to describe the canonical form of a word that has several variations for forming the plural or a particular tense. For common nouns, like "apple", this would be the singular form. For verbs, it is the untensed form (that is the one that would follow the word "to" in an infinitive, such as "to eat", "to be", or "to go"). Linguists use the term lexeme to describe a word type – which includes the set of the lemma and all its variants. The term morpheme is used to describe strings that carry meaning but may be smaller than a word, such as prefixes (which are substrings at the front of a word) and suffixes (which are substrings at the end of a word). Both can add either syntactic or semantic information to a word. Analyzing a word into morphemes is called "morphology". Finding the root is called lemmatization. NLP work sometimes uses the notion of "stems" instead of roots. Stems are substrings of a word, which can depend on an implementation, as there is no standard form. They are useful for specifying patterns to match all members of a lexeme. NLP work also uses the term "token", which is an instance of a word as it occurs in use. So, if a sentence includes the same word twice, there will be two separate tokens created for it.

NOUNS, PRONOUNS, AND PROPER NOUNS

Singular form	Regular plural form
frog	frogs
idea	ideas
fly	flies
fox	foxes
class	classes

Figure 3.1 Example of nouns with a plural formed with -s or -es suffix

Singular form	Irregular plural form
child	children
sheep	sheep
goose	geese
knife	knives

Figure 3.2 Example of nouns with an irregular plural

DETERMINERS

Determiners include "the", "a", "an", "that", "these", "this", and "those". There are also determiners that are used in questions, such as "what" and "which". Determiners only occur in noun phrases, before any adjectives or nouns. Some common nouns, when they express a mass quantity, like "water" or "rice", or when they are plural like "cats", do not require a determiner. Proper nouns generally do not allow a determiner, except when they are plural, e.g., "The Smiths" or when it is part of the name itself, e.g., "The Ohio State University". Possessive phrases, which are marked with an apostrophe and the suffix "-s", can take the place of a determiner, as in the phrase "my mother's house". Pronouns, regular or possessive, are *never* preceded by determiners. Determiners are considered a closed class of words.

VERBS AND AUXILIARY VERBS

Verbs are usually tensed (past, present, future). They include both verbs where the tensed forms are regular (see Figure 3.4) or irregular (see Figure 3.5). Also, in some contexts, verbs can appear untensed, such as after an auxiliary or after the word "to". Verbs are also marked for number (singular or plural), and for person. First person is "I"; second person is "you"; and third person is "he", "she", or "it". The third-person singular form is marked with "-s"; the non-3rd person singular present looks the same as the root form. Verbs also have participle forms for past (eg., "broken" or "thought") and present (e.g., "thinking").

PREPOSITIONS

Prepositions, such as “with”, “of”, “for”, and “from” are words that relate two nouns or a noun and a verb. Prepositions require a noun phrase argument (to form a prepositional phrase). It is estimated that there about 150 different prepositions (including 94 one-word prepositions and 56 complex prepositions, such as “out of”)[14]. Prepositions are generally considered a closed class, but the possibility of complex combinations suggests that algorithms might be better off allowing for out of vocabulary examples.

ADJECTIVES AND ADVERBS

Adjectives normally modify nouns, as in “the big red book”, but may also be an argument of a verb (including forms of “be”, “feel”, “appear”, and “become”). Adjectives can also be marked as comparative (meaning “more than typical”, using the suffix “-er”) or superlative (meaning “more than any others”, using the suffix “-est”). Adverbs modify verbs, adjectives, or other adverbs. They express manner or intensity. They may be comparative (e.g., “better”) or superlative (e.g., “best”). Adverbs that end in the suffix “-ly” have been derived from a related adjective (e.g., “quickly“ is derived from “quick“).

CONJUNCTIONS

Conjunctions, such as “and”, “although”, “because”, “but”, “however”, “or”, “nor”, “so”, “unless”, “when”, “where”, “while”, etc. are words that join words, phrases, clauses, or sentences. They can be discontinuous, e.g,. “either … or”, “neither … nor”, “both … and”, “not only … but also”, “on the one hand … on the other (hand)”, “not just … but”, and “not only … but”. They can take modifiers, such as “particularly”, as in Figure 3.6.

Example sentences	Type of expression with labels given by CoreNLP
You can walk but you cannot run.	Coordinating conjunction, labelled as conjunction
You can read at home or in the library.	Coordinating conjunction, labelled as conjunction
My daughter and her friends like to climb.	Coordinating conjunction, labelled as conjunction
My cat purrs when **you pet her**.	Subordinating conjunction, labelled as adverb
If *you study hard*, **you will do well**.	Subordinating conjunction, labelled as preposition
You can either walk or take the bus.	Correlative conjunction, labelled as adverb, conjunction
The car not only is quiet but also handles well.	Correlative conjunction, labelled adverb, adverb, conjunction, adverb
Embryonic Stem cells have a high mitotic index and form colonies. So, *experiments can be completed rapidly and easily.*	Discourse adverbial, labelled as adverb
Obese cats have higher levels of inflammatory chemicals in their bloodstream. However, *rapid shifts in fat tissue further increase this inflammation.*	Discourse adverbial, labelled as adverb

Figure 3.7 Examples of conjunctions

WH-WORDS

Wh-words that begin with the letters "wh-" like "who", "what", "when", "where", "which", "whose", and "why" and their close cousins "how", "how much", "how many", etc. They are used for posing questions and are thus sometimes called interrogatives**.** Unlike the word types mentioned so far, they can be determiners, adverbs, or pronouns (both regular and possessive), and so it is typical to see them marked as a special subtype of each. Identifying phrases that include wh-words is important, because they usually occur near the front in written text and fill an argument role that has been left empty in its normal position, as in "Which book did you like best?" In informal speech one might say "You left your book where?" or "You said what?", but the unusual syntax also suggests a problem (like mishearing, shock, or criticism). The semantics of the wh-expression specify what sort of answer the speaker is expecting (e.g., a person, a description, a time, a place, etc) and thus are essential to question-answering systems.

PART-OF-SPEECH TAGS

The general syntactic category of a word is also known as its part of speech (POS) whereas "tag" refers to labels for specifying the category and syntactic features (such as singular or plural). Today, the use of "tag" is often synonymous with the labels given in the Penn Treebank II (PT2) tagset[15],[16]. There was no process of agreement for adopting this tagset as a standard. Instead, a group of linguistic experts, with resources to support the work, developed them and disseminated them widely. Along the way, refinements have been made so that human annotation is more reliable and the sets of words in each category are not too sparse; early versions had about 80 tags, while the current only has only about 35. The tag set and associated guidelines for English have been stable since 2015. (The word tags have been stable since 1999). Figure 3.8 includes the complete English tagset for words, excluding punctuation.

CLAUSES AND SENTENCES

Clauses and sentences are structures that include a verb with a complete set of arguments. Sentences can either be statements (declaratives), questions (interrogatives), or commands (imperatives). Questions can either be yes-no questions (constructed by putting the auxiliary in front of the subject noun phrase) or wh-questions which include a question word at the front ("who", "what", "when", "where", "why", "how"), and sometimes also an auxiliary before the subject noun phrase. In wh-questions the question word takes the place of some other constituent in the sentence (either a subject or an object of a verb or preposition) that is an unknown, such as the type of something. A passive sentence is one where the semantic object appears as the syntactic subject (the one before the VP) as in "The apple was eaten". Some examples of different types of sentences are shown in Figure 3.10.

UNIT-1 (INTRODUCTION TO NLP)

LECTURE NO:-3&4

LEXICON, ALGORITHMS

LEXICON IN NLP

In the realm of Natural Language Processing (NLP), the analysis and understanding of human language play a crucial role. One of the fundamental components in NLP is the lexicon, which forms the building blocks for processing and interpreting textual data. The lexicon, also known as a vocabulary or wordbook, holds a significant position in NLP tasks, enabling algorithms to derive meaning from words in a computational manner. .

WHAT IS LEXICON?

The lexicon refers to the collection of words, phrases, or symbols in a specific language. It encompasses the vocabulary of a language and includes various linguistic attributes associated with each word, such as part-of-speech tags, semantic information, pronunciation, and more. It serves as a comprehensive repository of linguistic knowledge, enabling NLP systems to process and understand natural language text.

COMPONENTS OF A LEXICON

A lexicon comprises several components that provide rich information about words and their properties. These components include:

WORDS AND THEIR MEANINGS:

The core component of a lexicon is the listing of words, each associated with its corresponding meaning(s). This provides the fundamental building blocks for language understanding.

PART-OF-SPEECH (POS) TAGS:

POS tags assign grammatical categories to words, such as noun, verb, adjective, adverb, and more. POS tags play a vital role in syntactic analysis and help disambiguate word meanings based on their context.

PRONUNCIATION:

Lexicons often include information about the pronunciation of words, helping in tasks such as text-to-speech synthesis and speech recognition.

SEMANTIC INFORMATION:

Some lexicons include semantic attributes associated with words, such as word senses, synonyms, antonyms, and hypernyms. These semantic relationships enable algorithms to infer deeper meaning from text.

Here, lexicons in natural language programming refer to the group of words, as those of words or phrases are associated with certain features and in a certain style, such as the part of speech segmentation, as well as sentiment analysis, and they are also used as a source for interpreting human language by providing special information about meanings and uses, as well as grammatical properties those words since they contain information about the semantic relationship between words such as synonyms, antonyms, and hyper-expressions in that part, these lexicons can also be generated by hand automatically using machine learning, and they can also be of a multilingual nature.

IMPORTANCE OF LEXICONS

Dictionaries are an important resource in nervous linguistic programming for a number of reasons, as they provide information about the meanings of words and grammatical characteristics of words, as it is necessary for the interpretation and precise analysis of human life.

It is used on the feast from the famous and essential applications in the programming of the nervous abysses, such as refining feelings, translation, and detection of fraud messages and other applications. Here, without these dictionaries, these mentioned applications can be at risk.

Determine the patterns and the relationship between excessive words. This information is greatly necessary for a type of project such as text classification and also information retrieval process.

Reducing the arithmetic resources required for certain tasks, and this makes the efficiency of applications significantly improve, such as pre-hidden trends by the dictionary, the way that we can reduce the number of potential interpretations and focus on the most relevant interpretations on the required topic that is being worked on as we cannot exaggerate in the importance of dictionaries because it is considered a resource from Among the basic resources that support many major developments in the field of natural linguistic programming.

TYPES OF LEXICONS

A. SENTIMENT LEXICONS

In this dictionary, work is done to define the feelings as well as the feelings associated with a particular word or phrase. Where these dictionaries can specify a degree or label for each word indicating whether it is negative, positive, or neutral, this information is used to work on sentiment analysis, after which it

determines the general feeling of a part of the text, so we can see that in the part of the product review or a post on the media, The social media that is published and this part that we are talking about applies to, and we can also see it in the topic of celebrities and so on.

Examples of sentiment lexicons include the **AFINN lexicon** and the **SentiWordNet** dictionary.

B. EMOTION LEXICONS

These dictionaries are used in the special and specific part associated with words, as they are often used in emotion detection applications, such as chatbots or virtual assistants to understand the emotional state of the user who works on them and speaks to them or listens to them.

Examples of these dictionaries are **EmoLex** and **NRC** Emotion.

C. WORDNET

This is a lexical database that groups English words into groups of synonyms, called synsets, which they associate with conceptual relationships. These also include definitions, usage examples, and other semantic information for each synchronization. WordNet is often used as a reference for those NLP-specific tasks, such as recognizing named entities and also categorizing proposed text.

D. PROPBANK

This lexicon links verbs with their semantic arguments, as it provides information about the grammatical structure of the sentence and the roles that the different words play in that sentence. PropBank is used in many tasks such as classifying semantic roles and extracting information

What are NLP algorithms?

NLP algorithms are complex mathematical formulas used to train computers to understand and process natural language. They help machines make sense of the data they get from written or spoken words and extract meaning from them.

Put in simple terms, these algorithms are like dictionaries that allow machines to make sense of what people are saying without having to understand the intricacies of human language.

NLP algorithms use a variety of techniques, such as sentiment analysis, keyword extraction, knowledge graphs, word clouds, and text summarization, which we'll discuss in the next section

TYPES OF NLP ALGORITHMS

To help achieve the different results and applications in NLP, a range of algorithms are used by data scientists.

Some of the common types of NLP algorithms in data science include:

Sentiment analysis

Sentiment analysis is the process of classifying text into categories of positive, negative, or neutral sentiment.

It works through the use of several techniques:

1. TOKENIZATION

This is the first step in the process, where the text is broken down into individual words or "tokens". Each token is then analyzed separately.

2. STOP WORDS REMOVAL

Stop words such as "is", "an", and "the", which do not carry significant meaning, are removed to focus on important words.

3. TEXT NORMALIZATION

Words are converted to their base or root form. For example, "running" might be reduced to its root word, "run". This is known as Stemming or Lemmatization.

4. FEATURE EXTRACTION

Key features or words that will help determine sentiment are extracted from the text. These could include adjectives like "good", "bad", "awesome", etc.

5. CLASSIFICATION

The sentiment is then classified using machine learning algorithms. This could be a binary classification (positive/negative), a multi-class classification (happy, sad, angry, etc.), or a scale (rating from 1 to 10).

UNIT-1 (INTRODUCTION TO NLP)

Natural Language Processing

Data structures for implementation of the framework

LECTURE NO:- 5

The data structures most common to NLP are strings, lists, vectors, trees, and graphs. All of these are types of sequences, which are ordered collections of elements. Unprocessed data is usually input as string data which are processed into lists or vectors, representing individual words, before subsequent processing in the NLP pipeline. This processed data is usually not just a list or vector of strings, but sequences of complex objects that keep track of various attributes associated with each token, such as part of speech. Later stages may add additional annotations, such as marking the beginnings and endings of important sequences within a sentence, such as the names of entities. This section will overview the most commonly used data types along with examples of their application in NLP.

STRINGS

Strings are sequences of characters. They are the primary way that unprocessed text data is represented in NLP systems. One string may be used to hold an entire document, a line of a text file, a sentence, or a single word. Important operations for strings include concatenation (creating a string by sequentially combining two input strings into one), matching substrings (e.g., to check for a prefix or suffix), and finding the length of a string (e.g., strings that are very short might be abbreviations). Specifying patterns within strings, such as punctuation or word endings, is usually done by means of strings of text that allow one to name specific characters, possible ranges of characters (such as [a-z]), and how often they occur (e.g., once, zero or more, or one or more times). These formatted strings are called regular expressions and have been in use since the 1940's but have become more standardized over time, such as the POSIX Basic Regular Expression (BRE) format.[1][2]

LISTS

Lists and vectors are both ordered collection of elements. Lists are used to hold sequences where the sizes of elements might vary, such as sequences of words. Lists can also have more or less elements over time. (Lists that are immutable during the execution of a program are sometimes called tuples.) Some programming languages allow direct access to elements of a list using an index, while some may only allow iterating through them sequentially from the front. Lists (and more generally sequences) are often used as the representation format for sharing results between different stages of an NLP pipeline;

for example each element corresponds to all the information about a particular token in the order in which it occurred in the original input.

Pattern	Example of matching phrase

`[{"TEXT": {"REGEX": "^[Uu](\.?|nited)$"}},`
`{"TEXT": {"REGEX": "^[Ss](\.?|tates)$"}},`
`{"LOWER": "president"}]`
	United States president

`[{"ORTH": "("}, {"SHAPE": "ddd"}, {"ORTH": ")"},`
`{"SHAPE": "ddd"},{"ORTH": "-", "OP": "?"},`
`{"SHAPE": "dddd"}]`
	(414) 229-5000

`[{'LIKE_NUM': True, 'OP': '?'},`
`{'POS': 'ADJ', 'OP': '?'},`
`{'LEMMA': 'duck', 'POS': 'NOUN'}]`
	three yellow ducks

Figure 2.1 Examples of patterns to use with the spaCy matcher function

VECTORS

Vectors hold elements of the same size, such as numbers, and are of fixed size. They are one-dimensional, which means elements can be accessed using a single integer index. Similar representations of higher dimension are given special names; a matrix is a two-dimensional, rectangular structure arranged in rows and columns. The term tensor is used to describe generalizations of matrices to N-dimensional space.[7] Important operations for vectors include accessing the value of an element of the vector, finding the length of a vector, calculating the similarity between two vectors, and taking the average of multiple vectors.

THE VECTOR SPACE MODEL FOR INFORMATION RETRIEVAL

The Vector Space Model (VSM) [9] was introduced for automated information retrieval. In the VSM, the elements of the vector correspond to words in a vocabulary and the value is a weighted measure that combines the frequency of a term (word type) within a document with a measure of its frequency across a set of documents. The VSM draws on the insight that the importance of a word in a document is directly proportional to the number of times it appears in a document, but inversely proportional to the

number of documents in which it appears[10]. The combined measure is called "term frequency-inverse document frequency" (tf-idf). In this context, term frequency is the raw count of the term in a document divided by the total number of words in the document. Inverse document frequency is most commonly calculated as the log of the quotient of the total number of documents in the corpus divided by the number of documents that contain the term. The log is used to prevent calculation errors caused by numbers that are too small (called underflow). Variations of these measures are sometimes used to address data sparsity (called smoothing) or to prevent bias towards longer documents (normalization).

WORD EMBEDDINGS

Vectors are commonly used to represent meanings of words. These vector representations are called word embeddings. Embeddings differ from document vectors in that the components of word embeddings might not correspond to anything in the real world. Instead, these vectors organize words into a kind of continuous thesaurus or hash table, giving each word type a unique vector of numbers that allows us to tell words apart and also to measure their similarity (such as by the cosine measure discussed in the previous section). This idea has been explored most directly in an approach called hash embeddings[12].

TREES AND GRAPHS

When one considers the organization of an entire sentence, or processing architectures for finding such organization, NLP algorithms rely on the datatypes of trees and graphs. Graphs are specified by a set of nodes and connections between pairs of nodes, called edges. Trees are a special case of graphs where edges are directed edges and each node has a unique predecessor (the parent) and multiple possible successors (the children). By convention the root (a node with no predecessor in the tree) is drawn at the top. For NLP, data is most likely to be represented as a tree. Graphs are used as a processing model as a way of representing the state of a search or the architecture used to train a classifier.

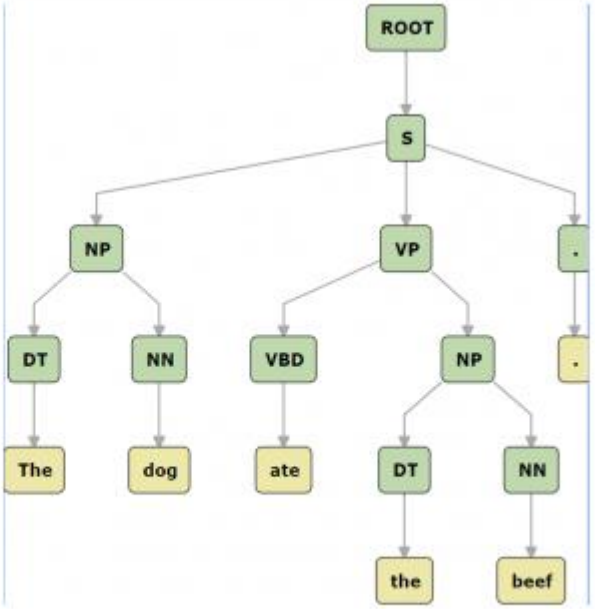


Figure 2.3 Example phrase structure parse tree (Image from Stanford CoreNLP.run)

UNIT-1 (INTRODUCTION TO NLP)

Natural Language Processing

Finite state automata.

LECTURE NO:- 6

FINITE AUTOMATA IN NATURAL LANGUAGE PROCESSING

Finite state automata (deterministic and non-deterministic finite automata) provide decisions regarding the acceptance and rejection of a string while transducers provide some output for a given input. Thus the two machines are quite useful in language processing tasks. Finite state automata are useful in deciding whether a given word belongs to a particular language or not.

Finite State Automata

The term automata, derived from the Greek word "αὐτόματα" meaning "self-acting", is the plural of automaton which may be defined as an abstract self-propelled computing device that follows a predetermined sequence of operations automatically.

An automaton having a finite number of states is called a Finite Automaton (FA) or Finite State automata (FSA).

Mathematically, an automaton can be represented by 5-tuple (Q, Σ, δ, q0, F), where −

► *Q is a finite set of states.*

► *Σ is a finite set of symbols, called the alphabet of the automaton*

► *δ is the transition function*

► *q0 is the initial state from where any input is processed (q0 ∈ Q)*

► *F is a set of final state/states of Q (F ⊆ Q)*

TYPES OF FINITE STATE AUTOMATION (FSA)

Finite state automation is of two types.

DETERMINISTIC FINITE AUTOMATION (DFA)

It may be defined as the type of finite automation wherein, for every input symbol we can determine the state to which the machine will move. It has a finite number of states which is why the machine is called Deterministic Finite Automaton (DFA).

Mathematically, a DFA can be represented by 5-tuple (Q, Σ, δ, q0, F), where −

► *Q is a finite set of states.*

► *Σ is a finite set of symbols, called the alphabet of the automaton.*

► *δ is the transition function where δ: Q × Σ → Q .*

► *q0 is the initial state from where any input is processed (q0 ∈ Q).*

► *F is a set of final state/states of Q (F ⊆ Q).*

Whereas graphically, a DFA can be represented by diagraphs called state diagrams where −

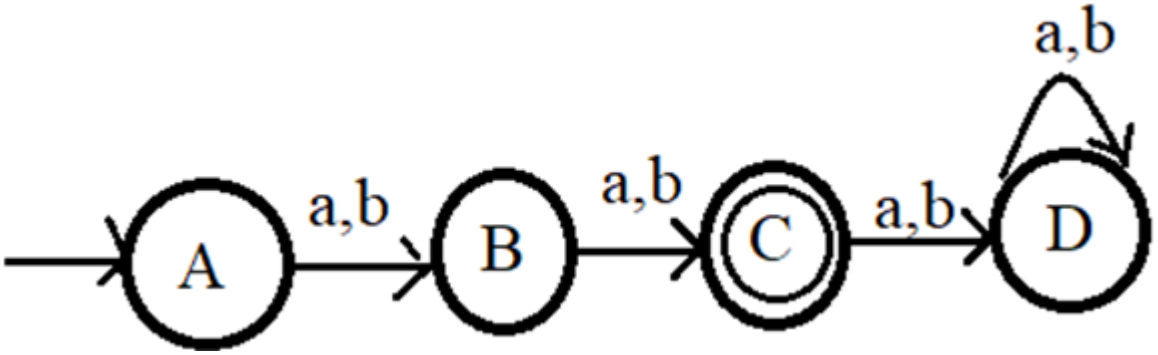
► *The states are represented by vertices.*

► *The transitions are shown by labeled arcs.*

► *The initial state is represented by an empty incoming arc.*

► *The final state is represented by double circle.*

Example of DFA —



NON-DETERMINISTIC FINITE AUTOMATION (NDFA)

It may be defined as the type of finite automation where for every input symbol we cannot determine the state to which the machine will move i.e. the machine can move to any combination of the states. It has a finite number of states which is why the machine is called Non-deterministic Finite Automation (NDFA).

Mathematically, NDFA can be represented by 5-tuple (Q, Σ, δ, q0, F), where −

► *Q is a finite set of states.*

► *Σ is a finite set of symbols, called the alphabet of the automaton.*

► *δ is the transition function where δ: Q × Σ → 2 Q.*

► *q0 is the initial state from where any input is processed (q0 ∈ Q).*

► *F is a set of final state/states of Q (F ⊆ Q).*

Whereas graphically (same as DFA), a NDFA can be represented by diagraphs called state diagrams where −

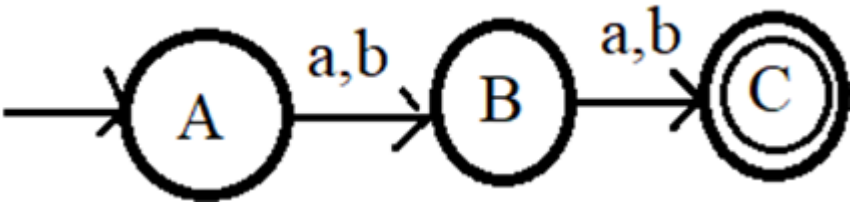
► *The states are represented by vertices.*

► *The transitions are shown by labeled arcs.*

► *The initial state is represented by an empty incoming arc.*

► *The final state is represented by double circle.*

Example of NDFA —



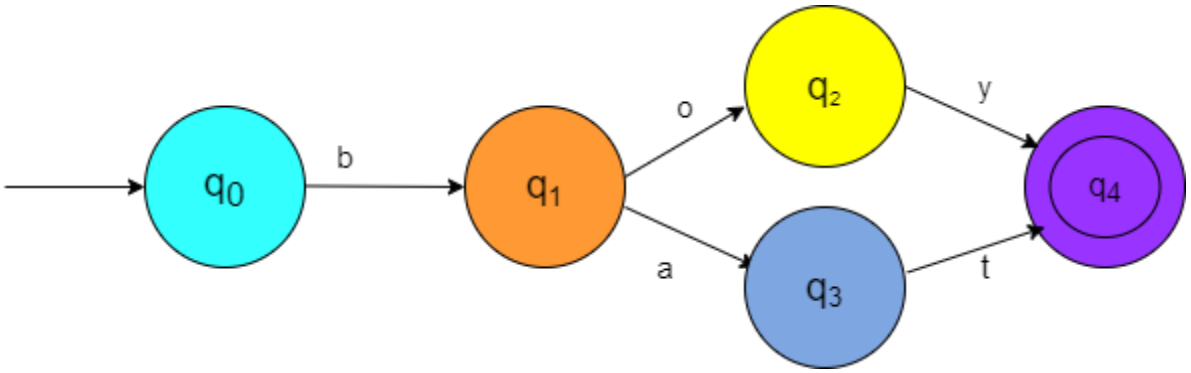
Finite Automata in NLP

FINITE STATE TRANSDUCER

A finite-state transducer (FST) is a finite-state machine with two memory tapes, following the terminology for Turing machines: an input tape and an output tape. This contrasts with an ordinary finite-state automaton, which has a single tape. An FST is a type of finite-state automaton (FSA) that maps between two sets of symbols. An FST is more general than an FSA. An FSA defines a formal language by defining a set of accepted strings, while an FST defines relations between sets of strings.

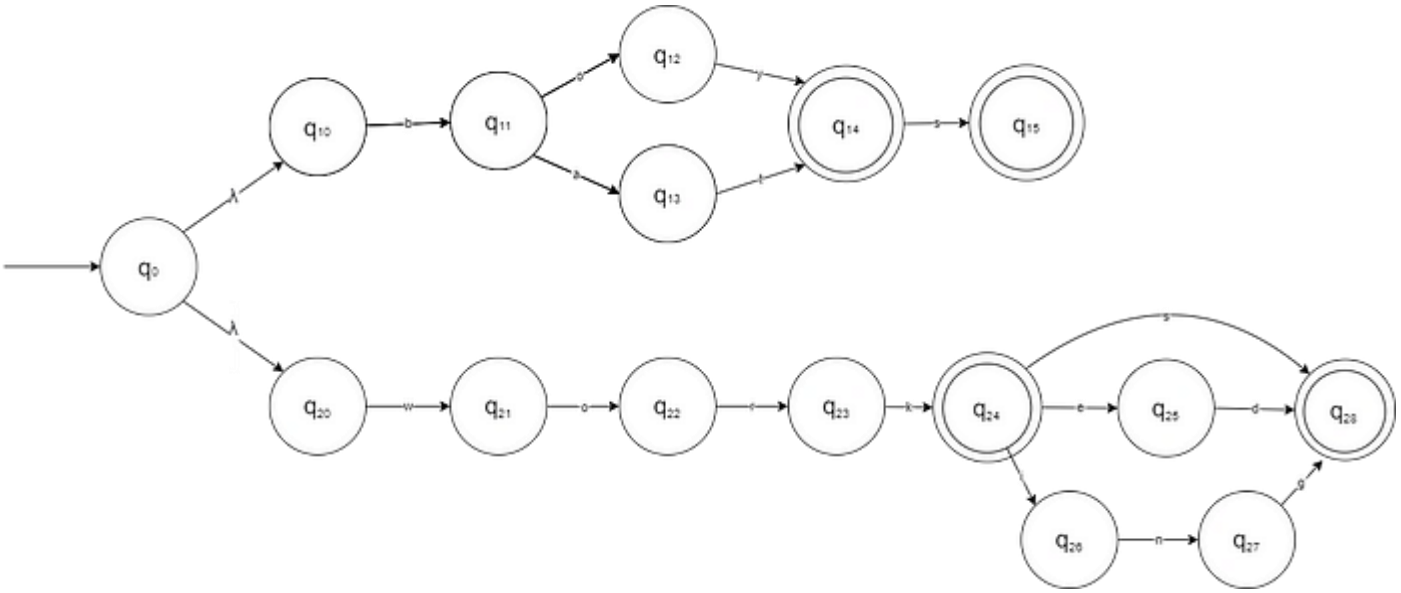
LANGUAGE RECOGNIZER

There are many tasks that need language recognizing mechanism. For example, spelling checker, morphological analysis, language identification etc.. Finite state machine are quite useful as a language recognizer. For a given word, a NFA can be designed easily that recognize the word. For example, NFA for the words 'boy' and 'bat' is shown in the Figure below. Similarly for every word a NFA can be designed and the different NFA's can be combined to form spelling checker or dictionary compilation for a language.



NFA for the words ‘boy’ and ‘bat’

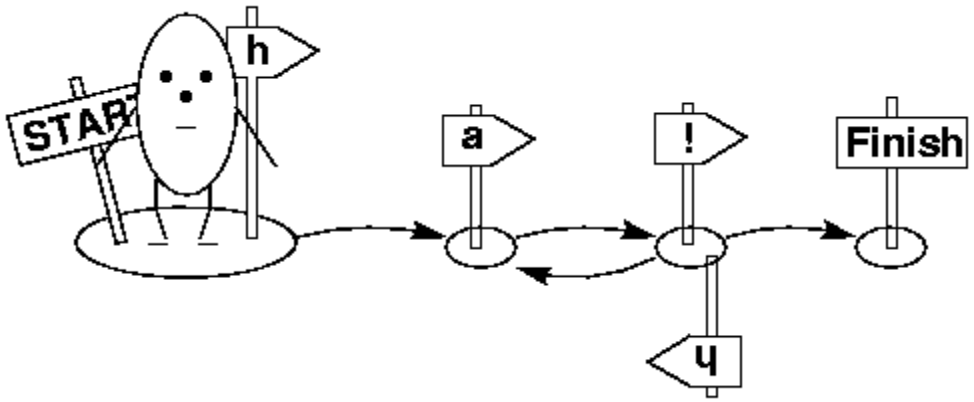
For each category of words, we can form a separate NFA and then combine them using transitions. For example, nouns and their plural can be recognized through one NFA and verbs and their different forms can be recognized through another NFA and finally, the two NFA can be combined. The figure below shows the NFA for some words and their morphological variations.



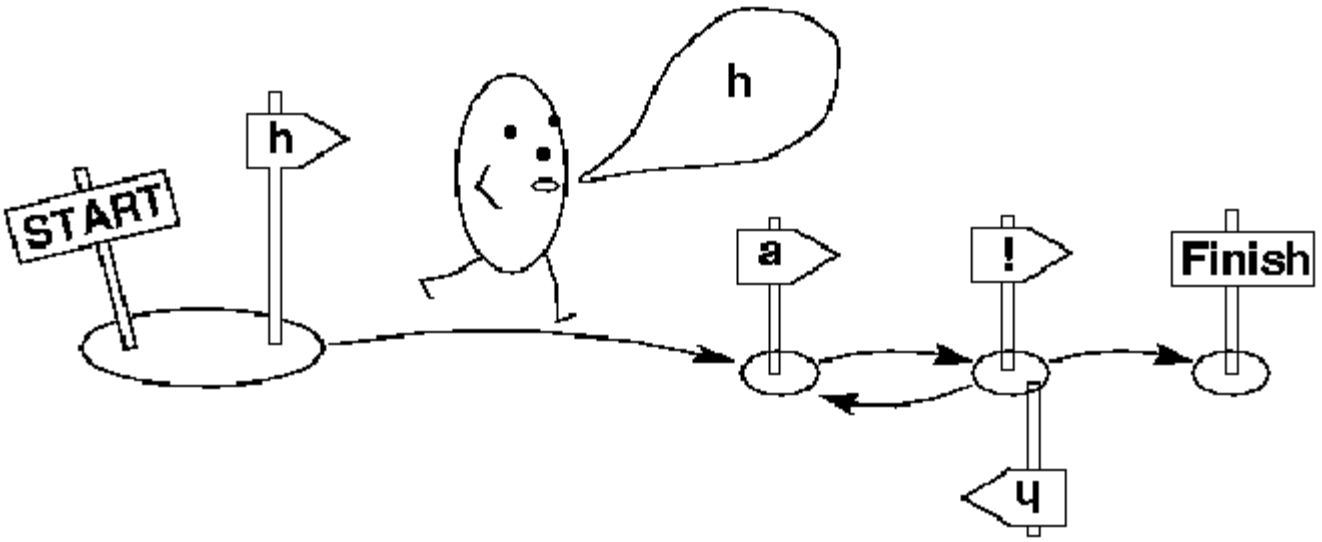
NFA for some words and their morphological variations

A SIMPLE MACHINE THAT CAN LAUGH

A finite state generator is a simple computing machine that outputs a sequence of symbols.

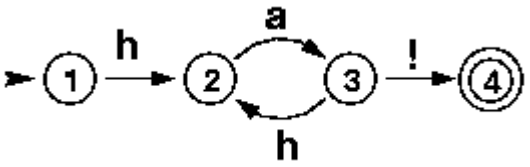


It starts in some start state and then tries to reach a final state by making transitions from one state to another. Every time it makes such a transition it *emits* (or *writes* or *generates*) a symbol.



So, what does the generator in the pictures say? It laughs. It generates sequences of symbols of the form ha! or haha! or hahaha! or hahahaha! and so on. Why does it behave like that? Well, it first has to make a transition emitting h. The state that it reaches through this transition is not a final state. So, it has to keep on going emitting an a. Here, it has two possibilities: it can either follow the ! arrow, emitting ! and then stopping in the final state (but remember, it can't look ahead to see that it would reach a final state with the ! transition) or it can follow the h arrow emitting an h and going back to the state where it just came from.

Finite state generators can be thought of as directed graphs. And in fact finite state generators are usually drawn as directed graphs. Here is our laughing machine as we will from now on draw finite state generators:



MORPHOLOGICAL PARSING AND GENERATION

Morphological parsing is the process of producing a lexical structure of a word, that is, breaking a word into stem and affixes and labeling the morphemes with category label. For example the word 'books' can be parsed as book+s and further as book + N+PL, the word 'went' can be parsed as go+V+PAST etc. generation is the reverse process of parsing, that is combining the lexical form of a word to produce the word. For example, box+N+Pl generates the word 'boxes'. Finite state transducers are quite useful in morphological parsing. Let us consider the lexicon form of regular inflectional 'girl +N +PL'. Figure

below shows the transducer which convert the lexicon form 'girl+N+PL' into the word 'girls'. Let us assume that x represents the word 'girl' for simplification purpose as for every regular singular noun the input and output will remain same in a transducer, that's why a variable x can be used for a noun. The word 'girl' can be replaced by any other regular noun like 'boy'.



Generation of word from its lexical form through a transducer

Here simple regular nouns have been considered on exemplarily basis, orthographic rules can be applied for irregular inflections and derivations and NFAs can be designed for every word and its variations in a certain language.

In other sense, we can say that morphology is the study of –

► *The formation of words.*

► *The origin of the words.*

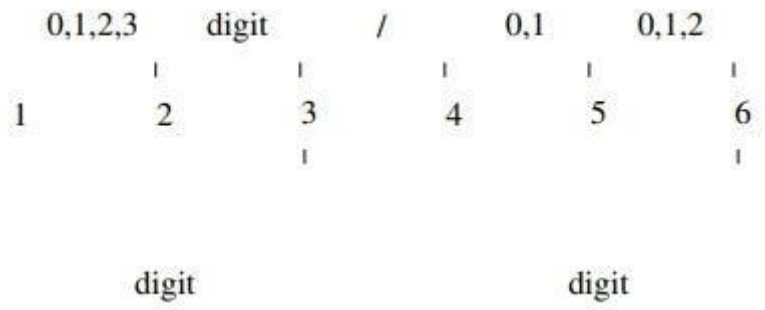
► *Grammatical forms of the words.*

► *Use of prefixes and suffixes in the formation of words.*

► *How parts-of-speech (PoS) of a language are formed.*

FINITE STATE AUTOMATA FOR RECOGNITION

The approach to spelling rules that is described here involves the use of finite state transducers (FSTs). Rather than jumping straight into this, we will briefly consider the simpler finite state automata and how they can be used in a simple recogniser. Suppose we want to recognise dates (just day and month pairs) written in the format day/month. The day and the month may be expressed as one or two digits (e.g. 11/2, 1/12 etc). This format corresponds to the following simple FSA, where each character corresponds to one transition:



This is a non-deterministic FSA: for instance, an input starting with the digit 3 will move the FSA to both state 2 and state 3. This corresponds to a local ambiguity: i.e., one that will be resolved by subsequent context. By convention, there must be no ‘left over’ characters when the system is in the final state.

UNIT-1 (INTRODUCTION TO NLP)

Natural Language Processing

Applications like machine translations.

LECTURE NO:- 7

MACHINE TRANSLATION

Machine Translation as discussed earlier, translates the meaningful text of one language to another language, with no human involvement. Machine Translation is evaluated on the BLEU(BiLingual Evaluation Understudy) score. BLEU is a metric for automatically evaluating machine-translated text. The score is between 0-1, the higher the score the better the machine translation.

Machine Translation has been present in the industry for several decades. Beginning in the 1970s, there were projects to achieve automatic translation. Over the years three major approaches have emerged:

- Rule-based Machine Translation (RBMT): 1970-1990
- Statistical Machine Translation (SMT): 1990-2010
- Neural Machine Translation (NMT): 2014-Present

RULE-BASED MACHINE TRANSLATION (RBMT): 1970-1990

RBMT systems are built on linguistic principles that allow words to be put in multiple locations and have varied meanings depending on the context. The RBMT approach is applied to the following language rules: transfer, analysis, and creation. Human language experts and programmers develop these rules.

Advantages:

- You do not need bilingual text
- Complete control (new law applicable in all cases)
- Reusable (existing language rules can be transferred when paired with new languages)

Disadvantages:

- Need good dictionaries
- Manually set rules (requires expertise)

Statistical Machine Translation (SMT): 1990-2010

SMT (Statistical Machine Translation) mainly uses or gets trained on existing human translation known as bilingual text corpus. As we have seen above the RBMT system mainly focuses on word-based translation but the SMT system focuses on phrase-based translation. The goal of phrase-based

translation is to remove the constraints of word-based translation by translating entire sequences of words of varying lengths. The word sequences are called phrases, however, they are generally not linguistic phrases, but rather phrases discovered using statistical approaches from bilingual text corpora.

Advantages:

- One SMT is suitable for two languages.
- Offline translation without a dictionary: with a proper language model, the translation is very smooth.

Disadvantages:

- Requires a bilingual corpus
- It is not appropriate for pairs of languages that have significant differences in word order.

NEURAL MACHINE TRANSLATION (NMT): 2014-PRESENT

NMT is a popular and widely used translation service that incorporates an end-to-end approach for automatic translation which overcomes the weaknesses of RBMT and SMT methods. NMT uses the most recent deep learning methods to produce better translation output than other traditional Machine Translation solutions. It is the most recent type of machine translation that employs a neural network that is closely related to the neurons of the human brain, allowing it to categorize data into various groups and layers. NMT is a language translation approach that tries to incorporate the context of the sentences or paragraphs rather than individual words. The NMT system is made up of current multilingual databases and automated learning mechanisms that contribute to continuous improvement.

Advantages:

- End-to-end models (no pipeline of specific tasks)

Disadvantages:

- Requires a bilingual corpus
- Sometimes unable to resolve unfamiliar words.

MACHINE TRANSLATION ALGORITHMS & MODELS

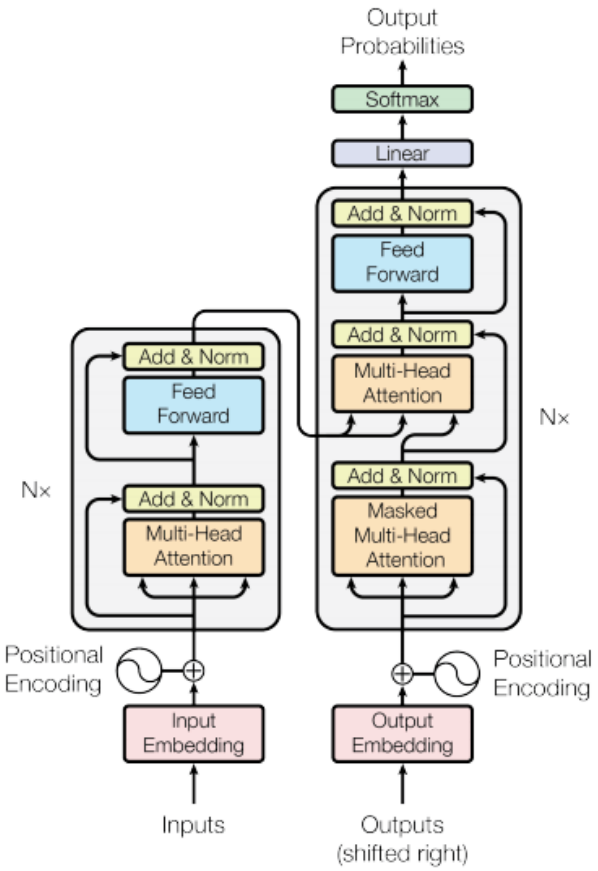
Without Encoder-Decoder models, Machine Translation algorithms would not exist or could not give better translations. Language translation models based on encoder-decoder machine translation have had a lot of success.

Mentioned below are some state-of-the-art models with an explanation of what encoder-decoder models are like:

- Attention is all you need
- Bidirectional Encoder Representation from Transformers (BERT)
- Generative Pre-trained Transformers (GPT)

Attention is all you need

"Attention is all you need" was proposed in 2017 and this was a huge breakthrough in the evolution of deep learning. The whole model architecture depends upon the attention mechanism. The architecture generally has an encoder-decoder structure. The encoder maps an input to generate the embedding with positional encoding and most importantly the attention mechanism. The decoder then generates an output with the help of a feed-forward neural network. The attention mechanism gives the flexibility to introduce more parameters to generate more features or relations between the data.



SOURCE

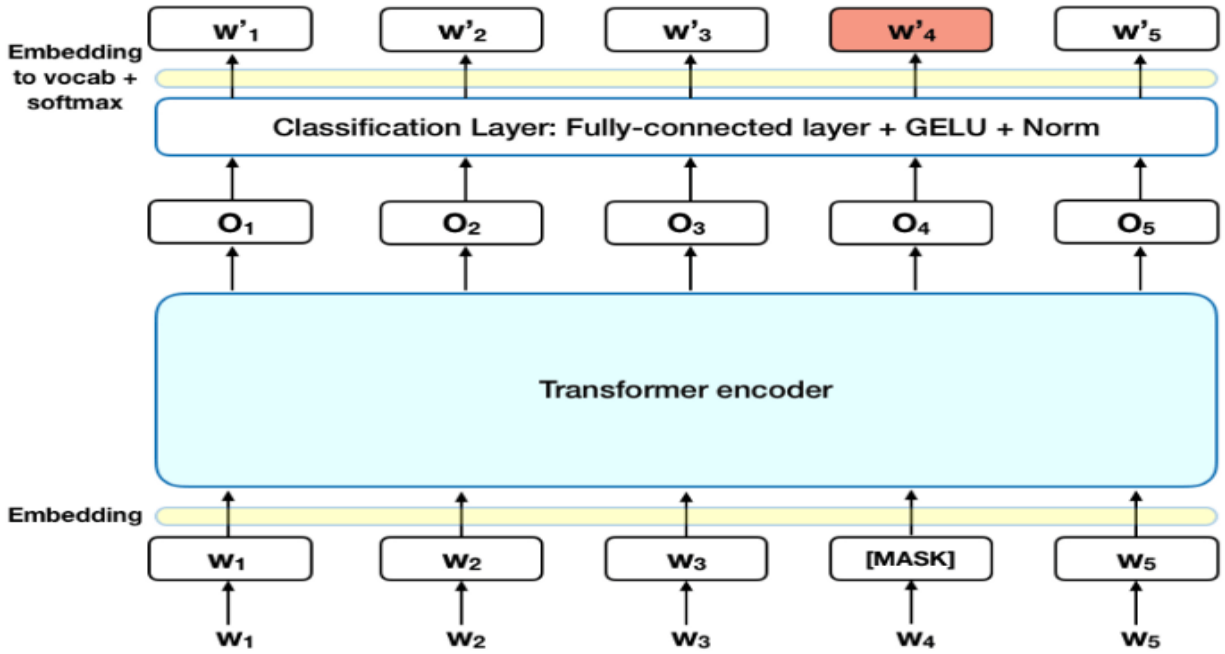
Encoder: In the above figure, the encoder is in the left which is composed identically in a stack of 6. The encoder has two sub-layers in each of them ie. multi-head attention with addition and normalization operation and another is a simple feed-forward network with addition and normalization operation with the skip connection between them.

Decoder: In the above figure, the right side is the decoder which is composed identically in a stack of 6. The decoder has three sub-layers, the two are identically the same as the encoder, the other layer is masked multi-head attention.

BIDIRECTIONAL ENCODER REPRESENTATION FROM TRANSFORMERS (BERT)

As the name suggests Bidirectional Encoder Representation from Transformers is an encoder-based architecture. The BERT model is based on the attention is all you need model, this is the reason "Attention is all you need" is the huge breakthrough in the evolution of deep learning. The BERT model can understand the meaning of complicated human languages in the text and perform various tasks like Machine Translation, Text Summarization, etc with a higher BLEU score and higher accuracy respectively.

BERT is a cutting-edge NLP model created by Google AI. BERT can understand the text in both directions and this operation is known as bi-directionality. BERT has also proposed Masked Language Model (MLM) training to hide some of the parts in the input, the learning or training of the model to predict the missing tokens.



Google Neural Machine Translation (GNMT)

Official Research Paper

The power of GNMT lies in its ability to read directly in the form of an end-to-end and map from input text to corresponding output text.

Its construction typically consists of two duplicate neural networks (RNNs), one to use input text sequences and one to produce translated text.

XLM model by Facebook

Official Github Official Research Paper XLM-R (New Model)

XLM-R is Facebook AI's latest cutting-edge XLM model for Machine Translation. XLM-R is a strong model that demonstrates the ability to train a single model for many languages while maintaining per-language performance. It has been trained on 2.5 TB of Common Crawl data in over 100 languages.

Generated Pre-trained Transformer (GPT)

You must have heard of the GPT category of language models, it has been developed by OpenAI. This phase, which includes the development of GPT-2 and GPT-3, has been attracting worldwide attention as they have been able to produce text similar to human-written text. As we have seen BERT is an encoder-based architecture based on transformers and GPT is a decoder-based architecture based on transformers. Both BERT and GPT are based on the "Attention is all you need" model. The full version of GPT-3 has 175 billion trainable parameters. GPT-3 can create poetry, articles, dialogues, stories, and it only needs a small amount of input text in the English language. GPT-3 also can create executable code for developers by just passing English text as input.

Multilingual Models Resource

Here you can find a good variety of multi-lingual models, pre-trained or tuned for several machine translations tasks and languages. Most of the multi-lingual models in the above mentioned link are state-of-the-art models with a high BLEU score

Benefits of Machine Translation

1. **Time efficient:** Machine Translation models can save a significant amount of time as they can translate a whole document in sec.
2. **Cost efficient:** It does not require human involvement which leads to lower cost.
3. **Memorizes terms:** Machine Translation models are designed in such a way that they memorize the key terms and reuses them wherever they fit.

Final Words

The sky is the limit when it comes to natural language processing. As technology becomes more widespread and greater advancements are explored, the future will see some major shifts in this domain. We hope this post helps you gain a better understanding of the NLP ecosystem as a whole.